

The Evolution of Transformers

From Softmax Attention to Kimi Linear Delta Attention

Askar Tsyganov

HSE University · Deep Learning 2

Lecture, 80 minutes

Who is talking

Askar Tsyganov

- MSc student in Applied Mathematics and Computer Science, HSE University
- Research Assistant, HDI Lab

Selected work

- *Matrix-Free Two-to-Infinity and One-to-Two Norms Estimation* — AAAI 2026
- *Bradley–Terry Rankings for Recommender Systems Across Dataset Taxonomies* — KDD 2026, oral
- *Faster SVD via Accelerated Newton–Schulz Iteration* — ICLR Blogposts 2026

fallnlove.github.io



Same formula, different machine

The operator has not changed since 2017:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Run it unchanged at $n = 10^6$ and the attention matrix alone is **2TB** — per head, per layer, before you multiply by V .

The question behind this entire lecture

How do you fit that much context into a model?

CONTEXT LENGTH

GPT-2, 2019
1 024

Llama-2, 2023
4 096

Kimi K3, 2026
1 000 000

What had to change?

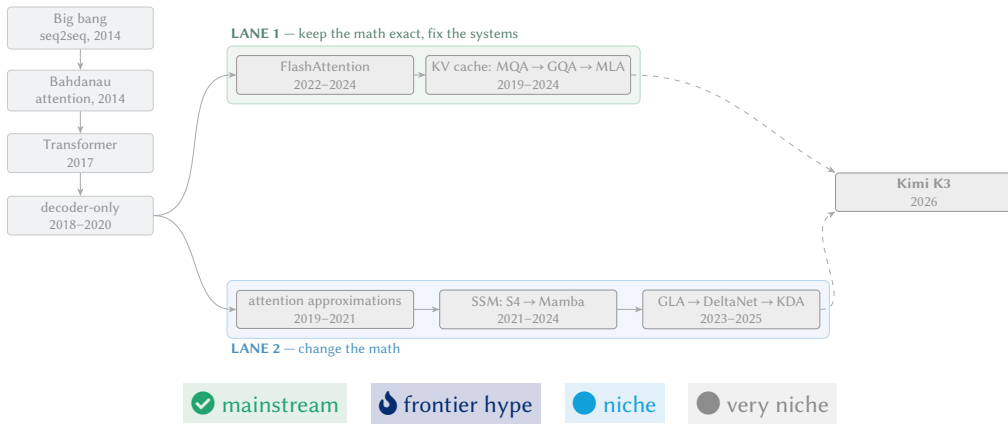
And the ask keeps growing



@Polymarket on X, 2026, quoting Sam Altman.

“Perfect context of your whole life” is a sequence that never stops growing. Whether or not the six months hold, that is the product roadmap of the whole industry — and every technique in this lecture exists to make such a sequence affordable.

Map of the lecture: one trunk, two lanes, one merge



The two lanes merge again on the right. That is not decoration — we hit the merge twice: at Mamba-2, and at Kimi K3.

Two lanes, two philosophies

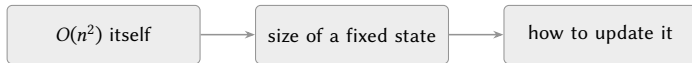
After 2018 everything happens *inside* a causal decoder-only model, and everyone is staring at the same quadratic bill. There are exactly two ways to react.

LANE 1 — mainstream
keep the math exact



FlashAttention, then MQA / GQA / MLA

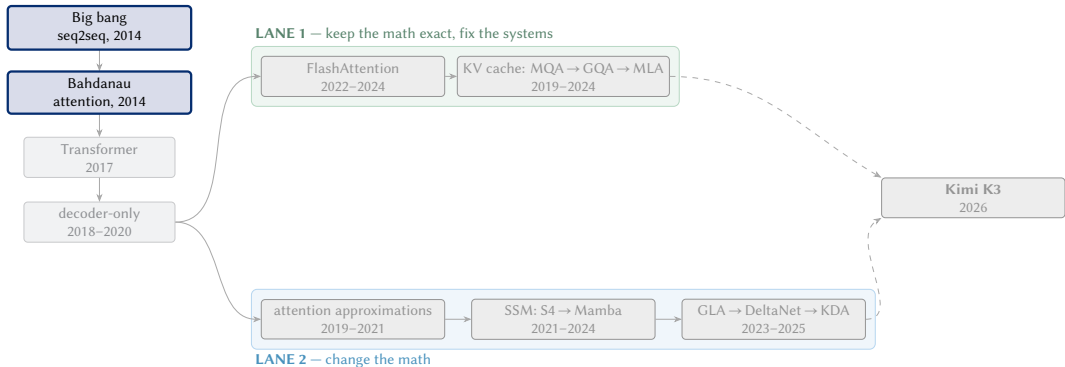
LANE 2 — niche
change the math



attention approximations, then SSM, then GLA / DeltaNet / KDA

↪ Each lane is told in chronological order — but when lane 1 ends in 2024 we **rewind to 2019** and walk lane 2 from its beginning.

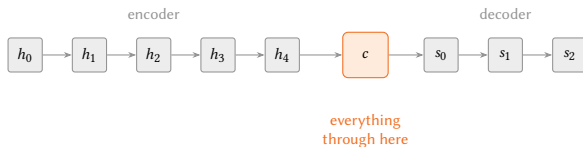
2014 — the primitive is invented



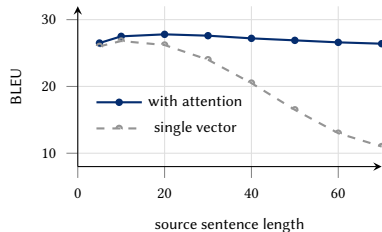
🎯 Bottleneck: **fixed-size sentence vector**

The problem: one vector has to hold the whole sentence

RNN encoder-decoder^{1,2} compresses the source into a single fixed-size c .



One bottleneck vector, any sentence length.



Schematic, after Bahdanau et al. (2014), Fig. 2.

¹Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. *Sequence to Sequence Learning with Neural Networks*. 2014. arXiv: [1409.3215](#).

²Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, et al. *Learning Phrase Representations using RNN Encoder-Decoder*. 2014. arXiv: [1406.1078](#).

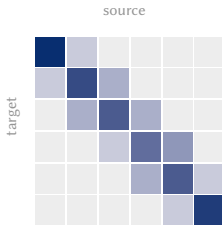
The fix: let the decoder look back, and learn where

Bahdanau, Cho and Bengio, 2014^a — one context vector *per output step*:

$$e_{ij} = v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$$

$$\alpha_{ij} = \frac{\exp e_{ij}}{\sum_k \exp e_{ik}} \quad c_i = \sum_j \alpha_{ij} h_j$$

- the weights are produced by a small MLP — **trained end-to-end**, with no alignment supervision anywhere;
- free side effect: α is a readable soft alignment.



Soft alignment α_{ij} (schematic).

^aDzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2014. arXiv: [1409.0473](#).

One year later: the score gets cheaper

Luong, Pham and Manning³ drop the MLP and try the obvious alternatives:

$$\text{score}(s, h) = \begin{cases} s^\top h & \textit{dot} \\ s^\top W h & \textit{general} \\ v^\top \tanh(W[s; h]) & \textit{concat (Bahdanau)} \end{cases}$$

- the MLP is replaced by a **dot product** — one matrix multiply instead of a per-pair nonlinearity;
- this is the exact operator that 2017 will scale up and build everything on.

↪ Still unsolved in 2015: attention was bolted *onto* an RNN. The recurrence — and its lack of parallelism — is still there.

³Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. *Effective Approaches to Attention-based Neural Machine Translation*. 2015. arXiv: [1508.04025](#).

What 2014 actually gave us — and what it charged for

The asset: content-based soft memory

- 1 information comes from the **whole history**, not from one compressed state;
- 2 **content decides** who attends to whom — not position, not a hand-designed structure;
- 3 access is soft $\Rightarrow$ differentiable $\Rightarrow$ learned jointly with the task.

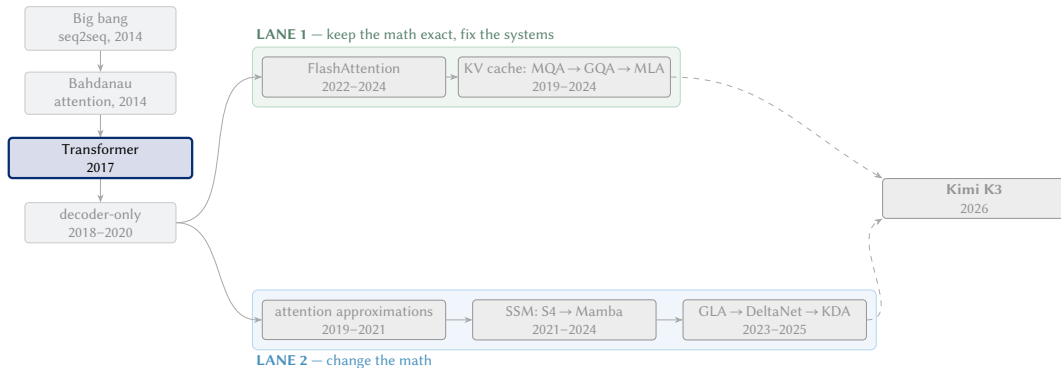
The debt: $n_{\text{enc}} \times n_{\text{dec}}$ weights

- cost grows as the **product of lengths**;
- nobody noticed in 2014 — the sequential RNN was the binding constraint, not the weight matrix;
- the bill arrives in 2019, when people want long context.

2014 wrote a cheque the industry is still paying.

~> Properties 1–3 are the invariant of this lecture: every later architecture keeps them while paying less. In the last block they reappear, applied to *network depth*.

2017 — recap only: you have read the paper



🎯 Bottleneck: **sequential training**

Why the recurrence had to go

Attention worked, but it was bolted onto an RNN, and an RNN cannot be parallelised over time.⁴

Layer type	Complexity / layer	Sequential ops	Max path length
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

Trading $O(n \cdot d^2)$ for $O(n^2 \cdot d)$ is a *win* as long as $n < d$. In 2017, with $n = 512$ and $d = 512$, it was.

ConvS2S and ByteNet bought the parallelism, but the path between tokens still grew with distance.

⁴Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. *Attention Is All You Need*. 2017. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762).

The price tag, computed out loud

One head, fp16: the attention matrix alone is $n^2 \times 2$ bytes.

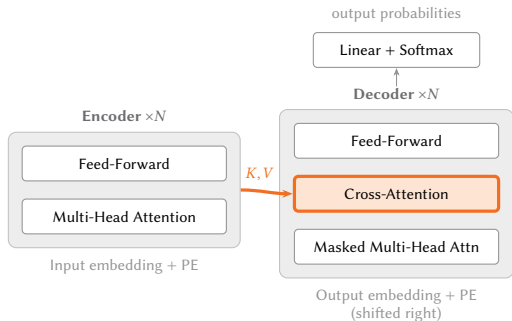
Sequence length	Attention matrix	
$n = 1\,024$	2 MB	nobody cares
$n = 8\,192$	128 MB	per head, per layer
$n = 65\,536$	8 GB	$\times$ heads $\times$ layers

Two properties of that operator we will lean on later: it is *permutation-invariant*, so position has to be injected from outside; and the causal mask is what makes incremental decoding — and therefore a KV cache — possible.

The one thing to carry out of this block

The Transformer traded “long memory + parallel training” for a **quadratic bill in n** . The remaining 60 minutes are eight different attempts to avoid paying it in full.

What the 2017 paper actually proposed



Schematic, after Vaswani et al. (2017), Fig. 1.

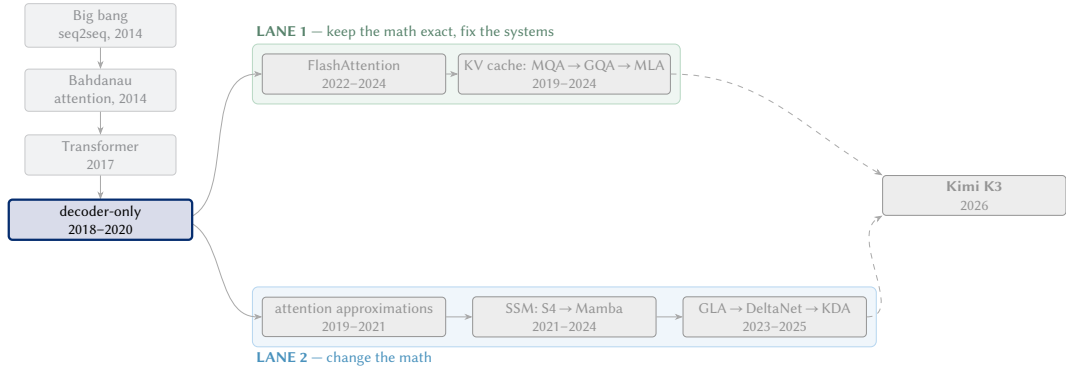
Two stacks and **three different uses** of the same operator:

- encoder self-attention — every token sees every token;
- decoder self-attention — *masked*, so a token never sees its own future;
- **cross-attention** — queries from the decoder, keys and values from the encoder. This is the only channel between the two halves.

Watch the orange arrow

One block from now the entire left half disappears, and that arrow with it.

2018–2020 — the fork that everything else sits on top of

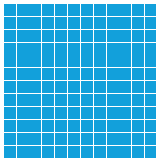


🎯 Bottleneck: **which stack to scale**

The fork: three families, three attention masks

Encoder-only

BERT, 2018^a



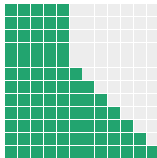
bidirectional
masked LM

great representations,
cannot generate

^aJacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2018. arXiv: [1810.04805](#).

Encoder-decoder

T5, 2019^a



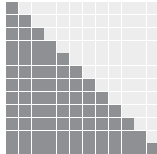
bidirectional prefix
+ causal target

natural for input → output,
two stacks to feed

^aColin Raffel, Noam Shazeer, Adam Roberts, et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2019. arXiv: [1910.10683](#).

Decoder-only

GPT-1/2/3^a



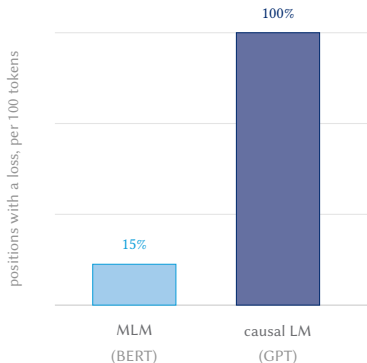
causal LM
one stack,

any text is training data

^aTom B. Brown, Benjamin Mann, Nick Ryder, et al. *Language Models are Few-Shot Learners*. 2020. arXiv: [2005.14165](#).

Same layers, same attention — only the allowed entries of the mask differ.

Argument 1: how much supervision does one pass buy?



- BERT-style MLM computes the loss on the $\approx 15\%$ of positions it masked. Causal LM computes it on **every** position: each token is a target once and context forever after;
- MLM also injects [MASK] symbols that never occur at inference — a train/test mismatch the causal objective simply does not have;
- no data curation: nothing has to be labelled “source” or “target”, so the training set is *the internet*;
- one objective, one loss curve $\Rightarrow$ scaling laws you can extrapolate before spending the compute.

Per unit of compute this is the cheapest gradient signal anyone had found.

Argument 2: the winner depends on what happens after pre-training

A controlled sweep over architecture $\times$ objective $\times$ adaptation⁵:

Evaluation regime	causal decoder	prefix (non-causal) decoder	enc-decoder
Zero-shot after pre-training	★ best	—	—
After multitask finetuning	—	—	★ best

- T5's own ablations agree: for *supervised* transfer, encoder-decoder with a denoising objective is the strong choice;
- so the 2019 answer and the 2021 answer are genuinely different — the question was never architecture alone.

The bet that settled it

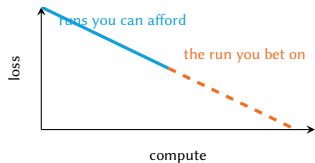
The industry bet on “**pre-train once, prompt for everything**” rather than “finetune per task”. GPT-3 made that bet pay: the task moves *into the prefix*. Given that bet, the top row is the one that matters.

⁵Thomas Wang, Adam Roberts, Daniel Hesslow, et al. *What Language Model Architecture and Pretraining Objective Work Best for Zero-Shot Generalization?* 2022. arXiv: [2204.05832](https://arxiv.org/abs/2204.05832).

Argument 3: scaling laws make it the bettable option

Test loss of an autoregressive LM follows smooth power laws in parameters, data and compute, over many orders of magnitude^a:

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}$$



^aJared Kaplan, Sam McCandlish, Tom Henighan, et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: [2001.08361](https://arxiv.org/abs/2001.08361).

- the point is not the exponent, it is **extrapolation**: fit on models you can afford, and you know the loss of one you have never trained. That is what lets an organisation commit a datacenter to a single run;
- this machinery was built for **one self-supervised objective on one stack** — the curve means the same thing at 100M and at 100B;
- a multitask encoder–decoder setup has a *task mixture* in it: change the mixture and the curve moves.

The shape of the argument

Not “decoder-only is provably better”, but “decoder-only is the option you can **bet** on”. At frontier budgets that is the same thing.

Honest footnote, and what changes from here on

Encoder-decoder is not dead

Machine translation, speech (Whisper), and a 2024–2025 line of work adapting decoder-only checkpoints back into encoder-decoder. Cross-attention also came back in multimodality (Flamingo-style resamplers). Bidirectional encoders still dominate embeddings and retrieval.

But the objective changed

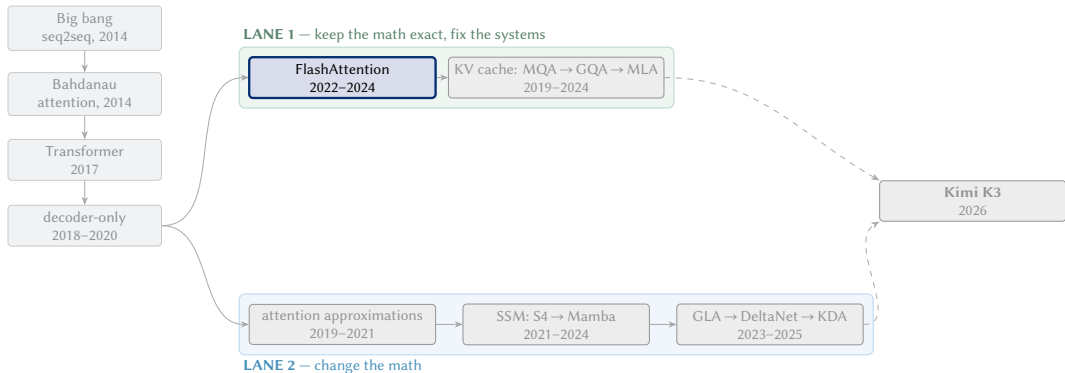
From here on everything in this lecture lives **inside** a causal decoder-only model. That is why it is on the trunk of our tree, not on a branch — and it reframes the problem: we now pay per **generated token**, not just per training step.



mainstream

the substrate every remaining branch is built on.

Lane 1 opens: 2022–2024 — the problem was never the FLOPs



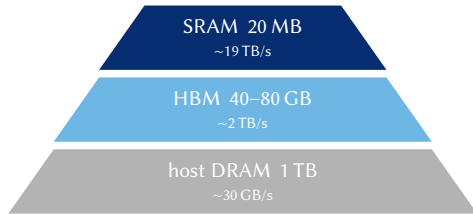
🎯 Bottleneck: **HBM bandwidth**

✅ Lane 1 · mainstream

Everyone was optimising the wrong quantity

Attention is **memory-bound**, not compute-bound. A naive implementation writes the $n \times n$ matrix to HBM and reads it back four or five times.

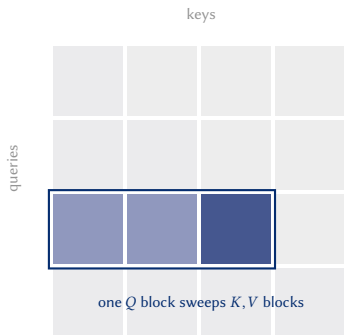
- the GPU can multiply faster than it can fetch;
- the metric that matters is **bytes moved**, not operations executed;
- three years of research had been minimising the wrong term.



NVIDIA A100, order of magnitude.

The winner is not whoever computes least. It is whoever *moves* least.

FlashAttention: tiling, online softmax, recomputation



- **tiling**: load blocks of Q, K, V into SRAM, finish the block there;
- **online softmax**: carry a running max and a running sum, so the full matrix never exists;
- **recomputation**: recompute attention in the backward pass instead of storing it.

$$m^* = \max(m, \tilde{m}), \quad \ell^* = e^{m-m^*} \ell + e^{\tilde{m}-m^*} \tilde{\ell}$$

The normaliser can be fixed up after the fact – that is the whole trick.

Same $O(n^2)$ FLOPs. Memory drops to $O(n)$, wall clock drops 2–4×, and the result is **exact**.⁶

⁶Tri Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. 2022. arXiv: [2205.14135](https://arxiv.org/abs/2205.14135).

Three more turns of the same crank, and one lesson

- **FlashAttention-2**[8]: better work partitioning across warps, $\approx 2\times$ over v1, up to $\sim 70\%$ of peak FLOPs on A100;
- **FlashAttention-3**[25]: Hopper asynchrony (TMA/WGMMA) and FP8, $\sim 75\%$ utilisation on H100;
- **FlashAttention-4**[9] (2026): Blackwell co-design — written in CuTe-DSL, deeper pipelining, 2-CTA MMA to cut shared-memory traffic, a hardware/software hybrid for the softmax exponentials. 1613 TFLOP/s, $\sim 71\%$ utilisation, $1.3\times$ over cuDNN — the first attention kernel past a petaflop.



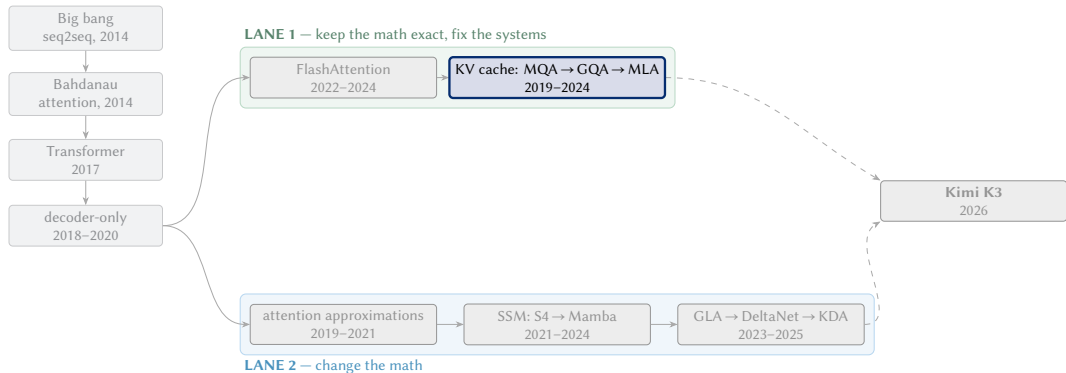
mainstream on by default, everywhere.

Methodological lesson

Before you change the model's mathematics, check whether you are actually bound by the memory hierarchy.

Lane 1's whole strategy in one line: **leave the mathematics alone, rewrite the kernel.** It also quietly destroyed a research programme — we meet the victims in lane 2.

2019–2024 — paying per generated token



🎯 Bottleneck: **KV cache size and traffic**

✅ Lane 1 · mainstream

What a KV cache is, and why you have no choice

Generation is one token at a time. At step t the layer has to compute

$$o_t = \text{softmax}\left(\frac{q_t K_{1:t}^\top}{\sqrt{d}}\right) V_{1:t}, \quad K_{1:t} = \begin{bmatrix} k_1 \\ \vdots \\ k_t \end{bmatrix}, \quad V_{1:t} = \begin{bmatrix} v_1 \\ \vdots \\ v_t \end{bmatrix}$$

Only q_t is new. And because the mask is causal, $k_i = W^K h_i$ and $v_i = W^V h_i$ depend on position i alone — **a new token never changes an old key or value.**

Option A — recompute

Rebuild $K_{1:t}, V_{1:t}$ from scratch at every step: $O(t d^2)$ per token, $O(n^2 d^2)$ per sequence. You push the entire prefix through every layer, once per generated token.

Option B — store

Compute each k_i, v_i once and keep them: $O(1)$ extra compute per token, at the price of $O(t d)$ memory per layer that **grows with every token you emit.**

Option B is the **KV cache**. It buys linear-time generation with memory — and the whole block is about how much memory.

And the bill for that memory

$$\text{KV bytes} = 2 \cdot n_{\text{layers}} \cdot n_{\text{kv heads}} \cdot d_{\text{head}} \cdot L \cdot \text{bytes/elem}$$

A 7B-class model: 32 layers, 32 heads,
 $d_{\text{head}} = 128$, fp16, $L = 32\text{k}$

$$2 \cdot 32 \cdot 32 \cdot 128 \cdot 32768 \cdot 2 \approx \mathbf{17 \text{ GB}}$$

per *single* request. At batch 8 the cache outweighs the weights.

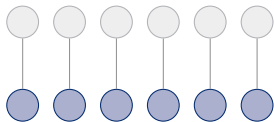
And it is read every step

Generating one token touches the entire cache.
So decoding is memory-bound too — the same disease FlashAttention cured, in a different organ.

This is why the next three ideas are not micro-optimisations: without them, long-context serving does not exist.

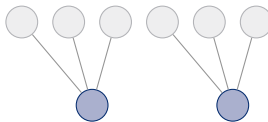
MQA and GQA: share the heads

MHA



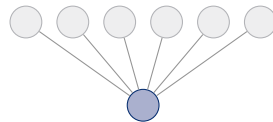
one KV head per query head

GQA



groups share KV

MQA



one KV head

- **MQA**^a: cache shrinks $n_{\text{heads}} \times$, quality visibly drops, training gets unstable;
- **GQA**^b: 4–8× with almost no loss $\Rightarrow$ the industry default.

Both are compromises

You buy memory with quality. The question DeepSeek asked in 2024: can we *stop* paying that price?

^aNoam Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. arXiv: [1911.02150](#).

^bJoshua Ainslie, James Lee-Thorp, Michiel de Jong, et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. arXiv: [2305.13245](#).

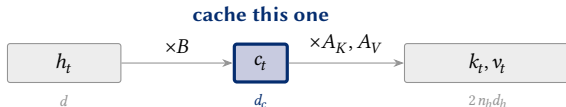
MLA is a low-rank factorisation of the K/V projections

Ordinary attention builds K and V with two big matrices, and caches the *results*:

$$k_t = W^K h_t, \quad v_t = W^V h_t, \quad W^K, W^V \in \mathbb{R}^{n_h d_h \times d} \Rightarrow 2 n_h d_h \text{ numbers per token}$$

DeepSeek-V2⁷ assumes these matrices are **low rank** and share the same right factor:

$$W^K = A_K B, \quad W^V = A_V B, \quad B \in \mathbb{R}^{d_c \times d}, \quad d_c \ll n_h d_h \Rightarrow k_t = A_K \underbrace{B h_t}_{c_t}, \quad v_t = A_V c_t$$



In the paper's notation $B = W^{DKV}$, $A_K = W^{UK}$, $A_V = W^{UV}$.

Method	cached / token / layer
MHA	$2 n_h d_h$
GQA	$2 n_g d_h$
MQA	$2 d_h$
MLA	$d_c + d_h^R \approx \frac{9}{2} d_h$

V2: $n_h=128, d_h=128, d_c=512 \Rightarrow 576$ vs 32 768.

⁷DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv: [2405.04434](https://arxiv.org/abs/2405.04434).

Why the latent is never decompressed

Naively you would recover $k_t = A_K c_t$ and $v_t = A_V c_t$ at every step — and pay back everything you saved. You never have to, because both factors can be moved to the **other side** of the product.

Scores: push A_K onto the query.

$$q_t^\top k_j = q_t^\top (A_K c_j) = \tilde{q}_t^\top c_j, \quad \tilde{q}_t := A_K^\top q_t$$

$\tilde{q}_t$ is formed *once per token*; c_j is the whole cache.

Output: pull A_V out of the sum, fold it into W^O .

$$\sum_j \alpha_j v_j = \sum_j \alpha_j A_V c_j = A_V \left(\sum_j \alpha_j c_j \right)$$

So K and V are **never materialised**: attention runs entirely in the d_c -dimensional latent space.

What it buys

- KV cache $\approx -93\%$ vs the previous DeepSeek generation;
- generation throughput $\times 5.76$;
- quality $\geq$ **MHA** — not a compromise, unlike MQA/GQA.

Same lesson as FlashAttention, one floor up: trade FLOPs for bytes and you win.

Where lane 1 goes next

NSA — Native Sparse Attention

DeepSeek, 2025^a: compression + selection + sliding window, *learned* rather than hand-designed, with kernels written for it from the start.

Note it for lane 2: sparse attention is back — but trainable and hardware-aligned. Hold that against the 2019–21 attempts we are about to see.

^aJingyang Yuan, Huazuo Gao, Damai Dai, et al. *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*. 2025. arXiv: [2502.11089](#).

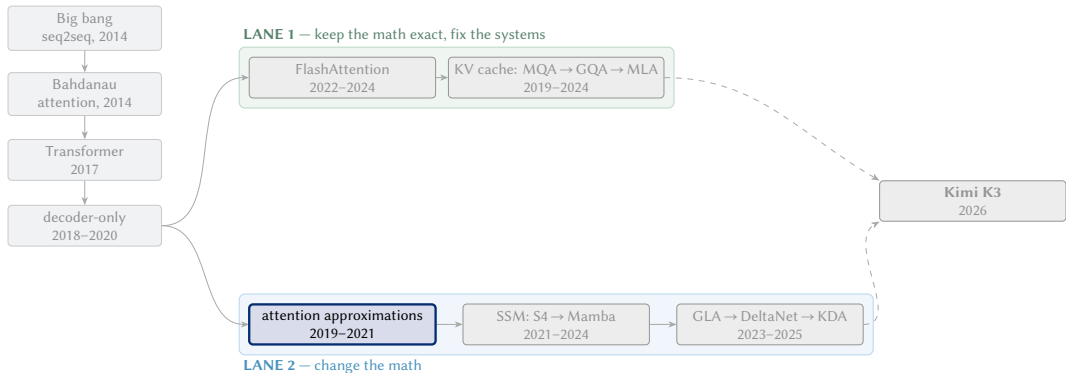
Lane 1, in one line

The operator never changed. What changed is **where the bytes live and how often they move**.

✓ mainstream

↪ Lane 1 ends here — exact attention, made cheap by systems work. Now rewind to 2019.

Lane 2 opens — rewind to 2019: approximate the softmax



🎯 Bottleneck: $O(n^2)$ **attention memory**

🧪 Lane 2 · niche

Three bets on the shape of one matrix

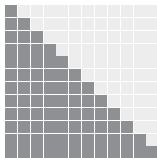
Rewind: it is 2019 again. FlashAttention does not exist, and the quadratic bill is the binding constraint. Lane 2's answer is the opposite of lane 1's — do not fix the kernel, **change the operator**.

Every method in this block is a bet about what the $n \times n$ attention matrix $A = \text{softmax}(QK^\top/\sqrt{d})$ really looks like:

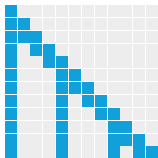
- 1 **A is sparse** — most pairs carry no mass, so fix a pattern in advance and skip the rest;
- 2 **A is low rank** — it can be written through a few hundred directions instead of n ;
- 3 **A is a kernel matrix** — and if that kernel factorises, the matrix never has to exist at all.

↪ We take them one at a time, then put them in one table. Watch what each bet costs when it is wrong.

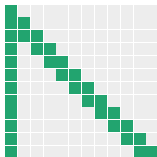
Bet 1 — sparsity: decide in advance who may look at whom



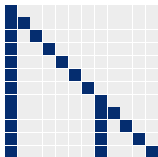
causal (full)



strided / fixed



window + global



window + random

- **Sparse Transformer**[5] (2019): strided and fixed patterns, factorised across heads — each head sees a slice, together they cover the matrix. $O(n\sqrt{n})$;
- **Longformer**[3] (2020): sliding window + dilated window + a handful of *global* tokens that everyone may attend to. $O(nw)$;
- **BigBird**[39] (2020): window + global + *random* edges. With theory attached: the resulting model is still a universal approximator of sequence functions and Turing complete.

What this bet costs when wrong

The pattern is a **prior you choose by hand**, identical for every input — exactly the choice attention was invented to make for itself.

Bet 2 — low rank: a few directions instead of n

- **Linformer**[31] (2020) projects K, V along the *sequence* axis with learned $E, F \in \mathbb{R}^{k \times n}$:

$$A V \approx \text{softmax}\left(\frac{Q(EK)^\top}{\sqrt{d}}\right)(FV), \quad O(nk)$$

Catch: E, F are tied to one fixed n , and you cannot project away a future you are not allowed to see — causal masking breaks.

- **Nystromformer**[33] (2021) picks m *landmarks* (segment means of Q and K) and reconstructs the matrix from them:

$$A \approx \tilde{F} \tilde{A}^+ \tilde{B}, \quad \tilde{F} = \sigma(Q\tilde{K}^\top), \quad \tilde{A} = \sigma(\tilde{Q}\tilde{K}^\top), \quad \tilde{B} = \sigma(\tilde{Q}K^\top), \quad O(nm)$$

No learned projection, so variable length and causality are no longer a problem.

- **MonarchAttention**[37] (2025) projects A onto *Monarch* matrices — products of block-diagonal factors with permutations: $\Theta(n\sqrt{nd})$ compute, $\Theta(nd)$ memory, and the conversion is **zero-shot** — no retraining, 4.5× over FlashAttention-2 at $n = 4k$.

Bet 3 — the kernel trick: what softmax actually is

Write attention for one query, without matrix notation:

$$o_i = \frac{\sum_{j \leq i} \exp(q_i^\top k_j / \sqrt{d}) v_j}{\sum_{j \leq i} \exp(q_i^\top k_j / \sqrt{d})} \implies o_i = \frac{\sum_{j \leq i} \kappa(q_i, k_j) v_j}{\sum_{j \leq i} \kappa(q_i, k_j)}$$

The exponential does one job: producing a **non-negative similarity** $\kappa(q, k)$ — nothing forces it to be exp.

Suppose now the kernel **factorises**, $\kappa(q, k) = \varphi(q)^\top \varphi(k)$ with $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$. Then the sum can be reassociated:

$$\sum_{j \leq i} v_j \varphi(k_j)^\top \varphi(q_i) = \underbrace{\left(\sum_{j \leq i} v_j \varphi(k_j)^\top \right)}_{S_i \in \mathbb{R}^{d \times d'}} \varphi(q_i) \implies \boxed{S_t = S_{t-1} + v_t \varphi(k_t)^\top}$$

The inner sum no longer depends on i : compute it once and carry it forward. $O(nd d')$ instead of $O(n^2 d)$, and **nothing is cached** — the state is a fixed $d \times d'$ matrix.

Choice of φ : $\text{elu}(x) + 1$ (Linear Transformer[17]) — cheap, but an approximation of nothing; positive random features (Performer[7]) — unbiased for exp, at the price of variance.

Now the table makes sense

Bet	Mechanism	Representatives	Cost	What broke
Sparse	fixed mask, chosen by hand	Sparse Transformer, Longformer, BigBird	$O(nw)$	the prior is the same for every input
Low rank	project or reconstruct from $k \ll n$ directions	Linformer, Nyströmformer, MonarchAttention	$O(nk)$	causal masking, variable length
Kernel	$\varphi(q)^\top \varphi(k)$, then reassociate	Linear Transformer, Performer	$O(n)$	φ only approximates exp

There were well over a hundred of these — the X-former era. Three ideas, and the survivors fit on one line.

Why the whole branch died anyway

- **Long Range Arena^a**: most methods lose accuracy, and the promised speedup often fails to materialise at the lengths people actually use ($n \leq 4k$);
- asymptotics were right, constants were wrong — irregular memory access, poor tensor-core utilisation;
- hand-designed sparsity throws away exactly what the model had learned to do by itself;
- and then FlashAttention, which we just walked through, removed the premise: exact attention was no longer slow.

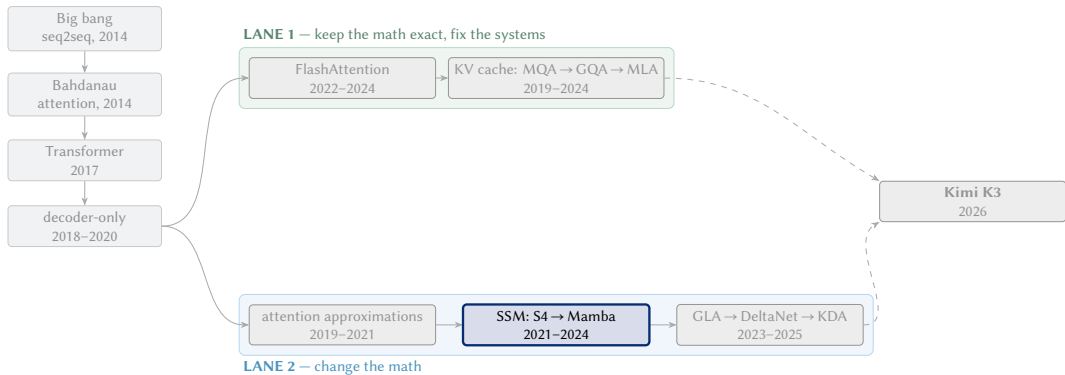
^aYi Tay, Mostafa Dehghani, Samira Abnar, et al. *Long Range Arena: A Benchmark for Efficient Transformers*. 2020. arXiv: [2011.04006](#).

What survived

Not as replacements for attention, but as components: sliding window (Mistral), attention sinks (StreamingLLM), *learned* sparsity in DeepSeek NSA, and structured matrices done properly in MonarchAttention.

● very niche

2021–2024 — keep a state, not a history



🎯 Bottleneck: **state of fixed size**

🧪 Lane 2 · niche

Same lane, a more radical move

A Transformer stores *everything*: the cache grows linearly and the cost per token grows with it. An RNN stores a fixed state — $O(1)$ per token — but it does not train in parallel.

The question this branch asks

Can we have the RNN's inference profile *and* the Transformer's training profile?

The answer starts somewhere unexpected: not from attention, but from a **linear dynamical system** borrowed from control theory.

$$h'(t) = A h(t) + B x(t), \quad y(t) = C h(t), \quad A \in \mathbb{R}^{N \times N}$$

h is a fixed-size summary of everything the signal has done so far. **S4**'s contribution⁸ is a structured A (HiPPO) for which that summary is provably a good compression of the history.

↪ But a differential equation cannot be run on a token sequence. It has to become an update rule first.

⁸Albert Gu, Karan Goel, and Christopher Ré. *Efficiently Modeling Long Sequences with Structured State Spaces*. 2021. arXiv: [2111.00396](https://arxiv.org/abs/2111.00396).

From a differential equation to an update rule

Pick a step size Δ — “how much time one token takes” — and ask where the state lands after Δ . For a linear system that has a closed form (zero-order hold):

$$\bar{A} = \exp(\Delta A), \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \Delta B$$

$$\implies \boxed{h_t = \bar{A} h_{t-1} + \bar{B} x_t, \quad y_t = C h_t}$$

The differential equation is gone; what is left is an **RNN update**.

Two readings worth stating

- Δ is a **knob on attention span**: $\Delta \rightarrow 0$ leaves the state untouched (ignore this token), large Δ refreshes it (focus on it);
- with A diagonal and negative, $\bar{A} = \text{Diag}(e^{\Delta a_1}, \dots)$ — **every channel of the state decays at its own rate**.

Why this trains fast (S4)

With $\bar{A}, \bar{B}, C$ fixed, unroll the recurrence:

$$y_t = \sum_{i \geq 0} C \bar{A}^i \bar{B} x_{t-i} \implies y = x * \bar{K}$$

$$\bar{K} = (C \bar{B}, C \bar{A} \bar{B}, C \bar{A}^2 \bar{B}, \dots)$$

One long convolution, evaluated by FFT: $O(n \log n)$ and **fully parallel**.

Mamba: make the system input-dependent

Selective SSM⁹: let Δ_t, B_t, C_t be *functions of* x_t . The step size is now chosen per token — the model decides what to let in and what to forget.

- the content-based gating idea from 2014, applied to a recurrence;
- but $\tilde{A}_t = \exp(\Delta_t A)$ now **changes every step**: the kernel is no longer fixed and the convolution trick dies;
- the recurrence is still **associative**, and that is enough:

$$(\tilde{A}_i, b_i) \oplus (\tilde{A}_j, b_j) = (\tilde{A}_j \tilde{A}_i, \tilde{A}_j b_i + b_j), \quad b_t = \tilde{B}_t x_t$$

so a **parallel scan** computes all states in $O(n)$ work and $O(\log n)$ depth;

- plus the systems half: fused kernels, state in SRAM, recomputation.

The second time in this lecture

Half the contribution is the idea. The other half is the kernel.

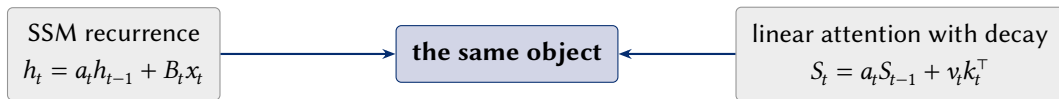
An architecture becomes real only when someone writes good CUDA for it.

$O(n)$ training, $O(1)$ memory per token, $\sim 5\times$ decode throughput, quality matching Transformers at the same size.

⁹Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2023. arXiv: [2312.00752](https://arxiv.org/abs/2312.00752).

Mamba-2: the two branches were the same animal

Structured State Space Duality¹⁰: take the transition matrix to be a scalar times identity, $\bar{A}_t = a_t I$. Then



- two communities, two vocabularies, one algorithm — and the SSD form runs on tensor cores, so Mamba-2 trains 2–8× faster than Mamba;
- every remaining slide in this lecture lives on the right-hand box.

¹⁰Tri Dao and Albert Gu. *Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality*. 2024. arXiv: [2405.21060](#).

Why pure SSMs stayed niche

A fixed state is an information bound

Mamba is strong on language modelling perplexity but weak exactly where you need to *retrieve*: multi-query associative recall, copying long strings, needle-in-a-haystack. Full attention is structurally better at this — it kept everything.

 **niche** as a pure architecture.

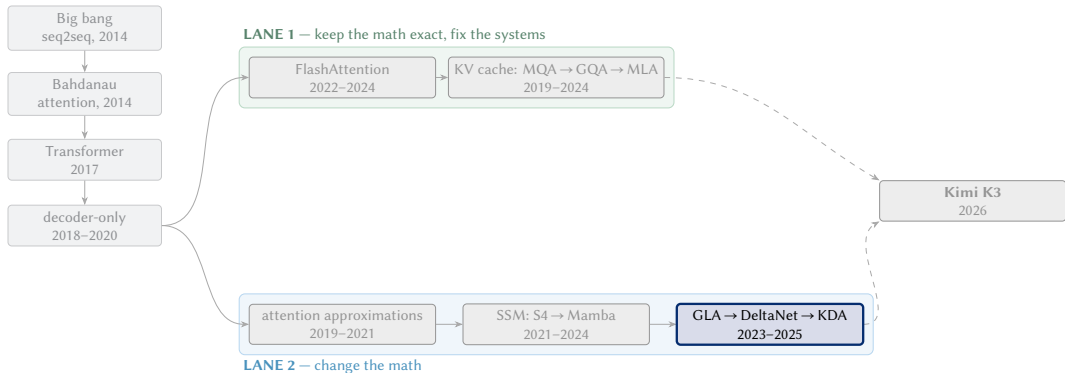
So: hybrids

Jamba, Samba, Zamba — insert a full-attention layer every N layers and the retrieval gap closes.

This is the shape of the answer for the rest of the lecture: **not replacement, composition.**

↪ Mamba-2 said SSM = linear attention. So there is only one question left: how should the state be updated?

2023–2025 — one line, six revisions



🎯 Bottleneck: **how to update the state**

🧪 Lane 2 · niche

Take softmax out and an RNN falls out

Ordinary causal attention, written for one query:

$$o_i = \frac{\sum_{j \leq i} \exp(q_i^\top k_j) v_j}{\sum_{j \leq i} \exp(q_i^\top k_j)} \xrightarrow{\text{drop exp}} o_i = \sum_{j \leq i} (q_i^\top k_j) v_j$$

Now the whole trick: $q_i^\top k_j$ is a *scalar*, so it commutes with v_j and the product can be regrouped —

$$\sum_{j \leq i} (q_i^\top k_j) v_j = \sum_{j \leq i} v_j (k_j^\top q_i) = \left(\underbrace{\sum_{j \leq i} v_j k_j^\top}_{S_i \in \mathbb{R}^{d \times d}} \right) q_i$$

This is an RNN

write $S_t = S_{t-1} + v_t k_t^\top$, read $o_t = S_t q_t$: a fixed $d \times d$ state, one rank-1 update per token. **Nothing is cached.**

	softmax	linear
state at step t	all past k, v	one matrix S
state size	$O(t d)$	$O(d^2)$
cost per token	$O(t d)$	$O(d^2)$

The cost simply stopped depending on t .

So the design question changes

The reframe that organises everything after 2023

Linear attention is an RNN whose hidden state is a **matrix** $S \in \mathbb{R}^{d \times d}$, and whose training still parallelises over chunks. So architecture design stops being about attention patterns. It is about one question:

what is the update rule for S ?

Reading is fixed forever: $o_t = S_t q_t$. Everything below differs only in how S is *written*.

Training does not run the recurrence token by token: inside a chunk it is plain matrix multiplication, only between chunks is it sequential. That is what makes these models trainable at scale.

And the naive rule is bad

$S_t = S_{t-1} + v_t k_t^T$ only ever **adds**. The state saturates, keys interfere, and nothing can ever be removed. Six years of work follow from that one sentence.

The ladder: five years, one line of math

Year	Model	Update rule for the state S	What it adds
2020	Linear Attention	$S_t = S_{t-1} + v_t k_t^\top$	$O(n)$, but never forgets

RetNet [27] · GLA [35] · Mamba-2 [10] · DeltaNet [36] · Gated DeltaNet [34] · KDA [21]

The ladder: five years, one line of math

Year	Model	Update rule for the state S	What it adds
2020	Linear Attention	$S_t = S_{t-1} + v_t k_t^\top$	$O(n)$, but never forgets
2023	RetNet	$S_t = \gamma S_{t-1} + v_t k_t^\top$	fixed exponential decay

RetNet [27] · GLA [35] · Mamba-2 [10] · DeltaNet [36] · Gated DeltaNet [34] · KDA [21]

The ladder: five years, one line of math

Year	Model	Update rule for the state S	What it adds
2020	Linear Attention	$S_t = S_{t-1} + v_t k_t^\top$	$O(n)$, but never forgets
2023	RetNet	$S_t = \gamma S_{t-1} + v_t k_t^\top$	fixed exponential decay
2024	GLA, Mamba-2	$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	a <i>learned</i> gate

RetNet [27] · GLA [35] · Mamba-2 [10] · DeltaNet [36] · Gated DeltaNet [34] · KDA [21]

The ladder: five years, one line of math

Year	Model	Update rule for the state S	What it adds
2020	Linear Attention	$S_t = S_{t-1} + v_t k_t^\top$	$O(n)$, but never forgets
2023	RetNet	$S_t = \gamma S_{t-1} + v_t k_t^\top$	fixed exponential decay
2024	GLA, Mamba-2	$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	<i>a learned gate</i>
2024	DeltaNet	$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	error-correcting write

RetNet [27] · GLA [35] · Mamba-2 [10] · DeltaNet [36] · Gated DeltaNet [34] · KDA [21]

The ladder: five years, one line of math

Year	Model	Update rule for the state S	What it adds
2020	Linear Attention	$S_t = S_{t-1} + v_t k_t^\top$	$O(n)$, but never forgets
2023	RetNet	$S_t = \gamma S_{t-1} + v_t k_t^\top$	fixed exponential decay
2024	GLA, Mamba-2	$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	<i>a learned gate</i>
2024	DeltaNet	$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	error-correcting write
2025	Gated DeltaNet	$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	gate <i>and</i> delta

RetNet [27] · GLA [35] · Mamba-2 [10] · DeltaNet [36] · Gated DeltaNet [34] · KDA [21]

The ladder: five years, one line of math

Year	Model	Update rule for the state S	What it adds
2020	Linear Attention	$S_t = S_{t-1} + v_t k_t^\top$	$O(n)$, but never forgets
2023	RetNet	$S_t = \gamma S_{t-1} + v_t k_t^\top$	fixed exponential decay
2024	GLA, Mamba-2	$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	<i>a learned gate</i>
2024	DeltaNet	$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	error-correcting write
2025	Gated DeltaNet	$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	gate <i>and</i> delta
2025	KDA (Kimi)	$S_t = S_{t-1} \text{Diag}(a_t) (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	channel-wise gate

All of 2025's “magic” is the evolution of a single line. Each row adds exactly one term.

RetNet [27] · GLA [35] · Mamba-2 [10] · DeltaNet [36] · Gated DeltaNet [34] · KDA [21]

Step 3: gating — learning to forget

Gated Linear Attention¹¹ replaces the scalar decay with a data-dependent, per-channel gate:

$$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top, \quad \alpha_t = \sigma(W_\alpha x_t)$$

- RWKV-5/6 and Mamba-2 are members of the same family;
- the real contribution is the **chunkwise** algorithm: recurrence between chunks, matrix multiplies inside them, so it runs on tensor cores.

Still not enough

The write is additive. You can fade the whole notebook, but you cannot go back and correct *one* entry.

¹¹Songlin Yang et al. *Gated Linear Attention Transformers with Hardware-Efficient Training*. 2023. arXiv: [2312.06635](#).

Step 4: the delta rule — writing by correcting the error

Treat the state as an associative memory and take *one gradient step at test time* on

$$\mathcal{L}_t(S) = \frac{1}{2} \|Sk_t - v_t\|^2:$$

$$S_t = S_{t-1} - \beta_t \nabla \mathcal{L}_t(S_{t-1}) = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

- read what is stored under k_t , compare with what *should* be there, correct by a fraction β_t ;
- roots in fast weight programmers^a; the 2024 contribution is a **parallel chunkwise form**^b — without it this does not train at scale;
- big jump on associative recall, exactly where GLA and Mamba were weak.

The notebook analogy

additive: append until you run out of paper
 · gate: pages fade uniformly · **delta**: find the right line and rewrite it.

^aImanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. *Linear Transformers Are Secretly Fast Weight Programmers*. 2021. arXiv: [2102.11174](#).

^bSonglin Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. 2024. arXiv: [2406.06484](#).

Step 5: Gated DeltaNet — do both

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

Gate for global housekeeping, **delta** for targeted overwrite.¹²

- beats Mamba-2 and DeltaNet on perplexity, long context and retrieval;
- shipped: Qwen3-Next is a Gated DeltaNet + full attention hybrid;
- RWKV-7 reaches a similar place from the DPLR side.^a

One knob still too coarse

α_t is a **scalar** per head. The entire memory fades at the same rate, no matter what is stored in which channel.

^aBo Peng, Ruichong Zhang, Daniel Goldstein, et al. *RWKV-7 “Goose” with Expressive Dynamic State Evolution*. 2025. arXiv: [2503.14456](#).

¹²Songlin Yang, Jan Kautz, and Ali Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2024. arXiv: [2412.06464](#).

Step 6: Kimi Delta Attention

$$S_t = S_{t-1} \text{Diag}(a_t) (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

Scalar gate → **channel-wise** gate: every channel of the state forgets at its own rate.¹³

- formally a special case of **DPLR** (diagonal-plus-low-rank) transitions, with the low-rank part tied to the keys — which buys a **bespoke chunkwise algorithm** far cheaper than general DPLR;
- interpretation worth stating out loud: the gate acts as *learned positional information inside the recurrence* — the role RoPE plays in attention.

Kimi Linear: 48B total / 3B active

- layerwise hybrid, **3 KDA : 1 MLA** — one layer from each lane, interleaved;
- −75% KV cache, up to **6×** decoding throughput at 1M context;
- first linear-attention architecture to **beat** a full-attention baseline under matched training.

¹³Moonshot AI (Kimi Team). *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025. arXiv: [2510.26692](#).

Status of the branch: hype, with one caveat

In production right now

- **Qwen3-Next** — Gated DeltaNet + attention
- **Kimi Linear** — KDA + MLA, 3:1
- **MiniMax-01** — lightning attention hybrid^a
- **Jamba** — Mamba + attention + MoE

^aMiniMax. *MiniMax-01: Scaling Foundation Models with Lightning Attention*. 2025. arXiv: [2501.08313](#).

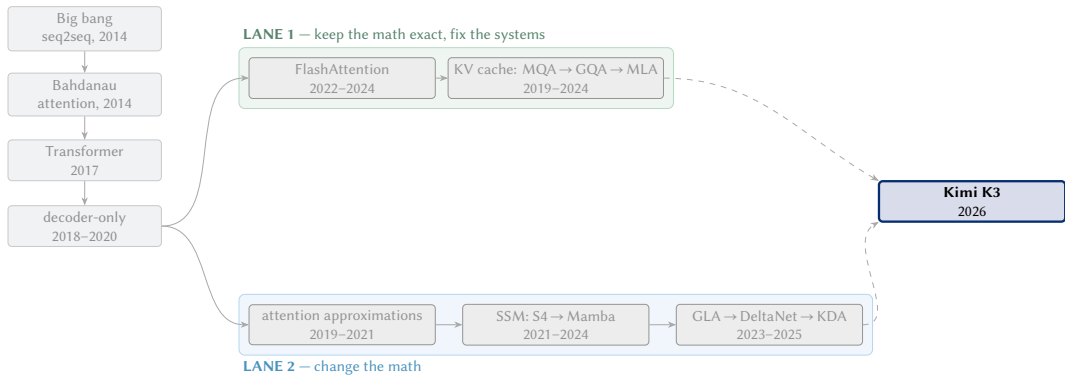
Read the list again

Every single one is a **hybrid**. Not one is pure linear attention.

Linear attention did not replace softmax attention. It became the cheap carrier of the load, with a few full-attention layers kept for exact retrieval.



2026 — the two lanes merge into one model



🎯 Bottleneck: **signal flow across depth**

✅ Lane 1 · mainstream

🧪 Lane 2 · niche

Kimi K3, in numbers

Released 16 July 2026; open weights 27 July 2026.¹⁴

Parameters	2.8T total / 104B activated per token
MoE	Stable LatentMoE: 896 experts, 16 active ($\approx 1.8\%$)
Attention	hybrid KDA + Gated MLA
Depth	AttnRes — attention residuals across layers
Context	1M tokens; native text / image / video
Precision	MXFP4 weights, MXFP8 activations, QAT from the SFT stage on
Claimed	$\approx 2.5\times$ scaling efficiency over Kimi K2; up to $6.3\times$ faster decoding at 1M

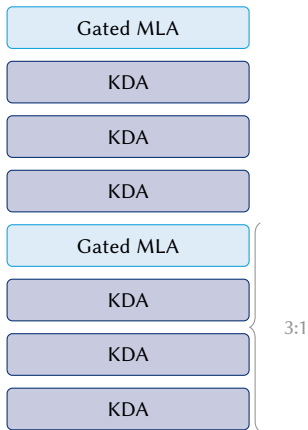
How to read the next four slides

You already know almost everything in that table. Name the block it came from.

¹⁴Moonshot AI. *Kimi K3 — technical blog*. 2026. URL: <https://www.kimi.ai/blog/kimi-k3>.

Component 1: KDA + Gated MLA

(one from each lane)



One layer from each lane, interleaved: lane 2's linear layer with a channel-wise gate carries the load, lane 1's latent attention keeps exact retrieval alive.

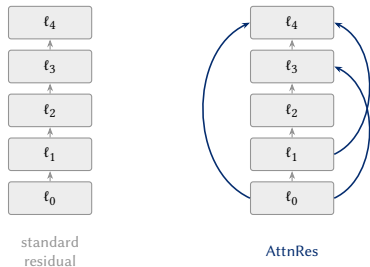
Two engineering deltas over Kimi Linear:

- **scaled sigmoid decay** in the KDA gate — numerical stability at 2.8T;
- **a gate added to MLA** itself, for selectivity.

Note the pattern

The distance between a paper and a frontier model is usually a handful of stabilisations, not a new idea.

Component 2: AttnRes — the one thing that is new



A layer may attend to representations from **all** preceding layers — **attention along depth** instead of along the sequence.

- why: at this scale a useful representation may have formed much earlier, and the signal struggles to survive the stack;
- cost: layers are partitioned into blocks of size N , taking memory and communication from $O(Ld)$ to $O(Nd)$;
- reported: $\approx +25\%$ training efficiency for $< 2\%$ overhead.

The punchline of the whole lecture

The 2014 primitive again — **soft, content-based addressing** — on a new axis. Bahdanau addressed source positions; the delta rule addresses memory slots; the MoE router addresses width; AttnRes addresses depth.

Component 3: Stable LatentMoE, and the byte budget

896 experts, 16 active

Extreme sparsity ($\approx 1.8\%$) held together by quantile balancing of the router and fully balanced expert parallelism with static shapes.

This is an *orthogonal* axis to our story — sparsity across **width**, not across **time**. It deserves its own lecture.

MXFP4 / MXFP8

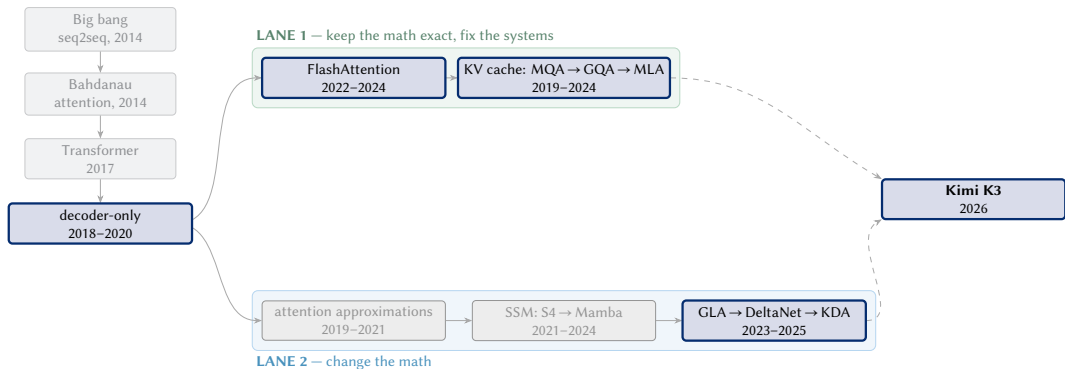
Quantization-aware training from the SFT stage onward: 4-bit weights, 8-bit activations.

Back to the FlashAttention lesson — at frontier scale, a large share of the engineering is not about the mathematics of the model but about **how many bytes you move**.

Where the verified numbers stop

The KDA:MLA layer ratio and the total layer count are **not disclosed** in the open material at the time of this lecture. Everything on these slides that is not in the table comes from the technical blog and secondary write-ups — treat it accordingly.

And here is the whole tree, inside one model



Evolution did not end with one lane winning. Lane 2 spent six years being “niche” — and then **graduated into lane 1’s models.**

The whole thing on one table

Approach	Training	Decode memory	Cost / token	Status 2026
Softmax attention + Flash	$O(n^2)$	$O(n)$ cache	grows with n	✓ mainstream
MQA / GQA / MLA	$O(n^2)$	$O(n)$, 10–20× smaller	grows with n	✓ mainstream
Attention approximations	$O(n)$	$O(n)$	grows with n	● very niche
SSM / Mamba	$O(n)$	$O(1)$	constant	● niche
Gated linear / DeltaNet / KDA	$O(n)$	$O(1)$	constant	🔥 frontier hype
Hybrids (Kimi K3, Qwen3-Next)	mixed	$O(1)$ on most layers	near constant	✓ mainstream

Five things to take away

- 1 Every architecture here is an answer to a specific bottleneck — not a beautiful idea looking for a use.
- 2 **The bottleneck moves**: translation quality $\rightarrow$ training parallelism $\rightarrow O(n^2)$ memory $\rightarrow$ HBM bandwidth $\rightarrow$ KV cache $\rightarrow$ state size and depth.
- 3 A systems optimisation can kill an entire research direction (FlashAttention vs. the X-formers). Profile before you redesign.
- 4 The branches **converge**: SSM = linear attention = fast weights = online learning of an associative memory. And in production they converge too — a hybrid, not a winner.
- 5 The 2014 primitive never left. **Soft content-based addressing** just keeps moving to new axes: sequence (attention), fixed-size memory (delta rule), width (MoE routing), depth (AttnRes).

If a fixed-size state always loses something,
and full attention is always quadratic —

what should the memory hierarchy of a model look like?

References I

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. arXiv: [2305.13245](#).
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2014. arXiv: [1409.0473](#).
- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. *Longformer: The Long-Document Transformer*. 2020. arXiv: [2004.05150](#).
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. *Language Models are Few-Shot Learners*. 2020. arXiv: [2005.14165](#).
- [5] Rewon Child et al. *Generating Long Sequences with Sparse Transformers*. 2019. arXiv: [1904.10509](#).

References II

- [6] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, et al. *Learning Phrase Representations using RNN Encoder–Decoder*. 2014. arXiv: [1406.1078](#).
- [7] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, et al. *Rethinking Attention with Performers*. 2020. arXiv: [2009.14794](#).
- [8] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. 2023. arXiv: [2307.08691](#).
- [9] Tri Dao, Ganesh Bikshandi, Jay Shah, et al. *FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling*. 2026. arXiv: [2603.05451](#).
- [10] Tri Dao and Albert Gu. *Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality*. 2024. arXiv: [2405.21060](#).
- [11] Tri Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. 2022. arXiv: [2205.14135](#).

References III

- [12] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. arXiv: [2405.04434](#).
- [13] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2018. arXiv: [1810.04805](#).
- [14] Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2023. arXiv: [2312.00752](#).
- [15] Albert Gu, Karan Goel, and Christopher Ré. *Efficiently Modeling Long Sequences with Structured State Spaces*. 2021. arXiv: [2111.00396](#).
- [16] Jared Kaplan, Sam McCandlish, Tom Henighan, et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: [2001.08361](#).
- [17] Angelos Katharopoulos et al. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. 2020. arXiv: [2006.16236](#).

References IV

- [18] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. *Effective Approaches to Attention-based Neural Machine Translation*. 2015. arXiv: [1508.04025](#).
- [19] MiniMax. *MiniMax-01: Scaling Foundation Models with Lightning Attention*. 2025. arXiv: [2501.08313](#).
- [20] Moonshot AI. *Kimi K3 — technical blog*. 2026. URL: <https://www.kimi.ai/blog/kimi-k3>.
- [21] Moonshot AI (Kimi Team). *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025. arXiv: [2510.26692](#).
- [22] Bo Peng, Ruichong Zhang, Daniel Goldstein, et al. *RWKV-7 “Goose” with Expressive Dynamic State Evolution*. 2025. arXiv: [2503.14456](#).

References V

- [23] Colin Raffel, Noam Shazeer, Adam Roberts, et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2019. arXiv: [1910.10683](#).
- [24] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. *Linear Transformers Are Secretly Fast Weight Programmers*. 2021. arXiv: [2102.11174](#).
- [25] Jay Shah, Ganesh Bikshandi, Ying Zhang, et al. *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision*. 2024. arXiv: [2407.08608](#).
- [26] Noam Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. arXiv: [1911.02150](#).
- [27] Yutao Sun, Li Dong, Shaohan Huang, et al. *Retentive Network: A Successor to Transformer for Large Language Models*. 2023. arXiv: [2307.08621](#).
- [28] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. *Sequence to Sequence Learning with Neural Networks*. 2014. arXiv: [1409.3215](#).

References VI

- [29] Yi Tay, Mostafa Dehghani, Samira Abnar, et al. *Long Range Arena: A Benchmark for Efficient Transformers*. 2020. arXiv: [2011.04006](#).
- [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. *Attention Is All You Need*. 2017. arXiv: [1706.03762](#).
- [31] Sinong Wang, Belinda Z. Li, Madian Khabsa, et al. *Linformer: Self-Attention with Linear Complexity*. 2020. arXiv: [2006.04768](#).
- [32] Thomas Wang, Adam Roberts, Daniel Hesslow, et al. *What Language Model Architecture and Pretraining Objective Work Best for Zero-Shot Generalization?* 2022. arXiv: [2204.05832](#).
- [33] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, et al. *Nyströmformer: A Nyström-Based Algorithm for Approximating Self-Attention*. 2021. arXiv: [2102.03902](#).

References VII

- [34] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2024. arXiv: [2412.06464](#).
- [35] Songlin Yang et al. *Gated Linear Attention Transformers with Hardware-Efficient Training*. 2023. arXiv: [2312.06635](#).
- [36] Songlin Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. 2024. arXiv: [2406.06484](#).
- [37] Can Yaras et al. *MonarchAttention: Zero-Shot Conversion to Fast, Hardware-Aware Structured Attention*. 2025. arXiv: [2505.18698](#).
- [38] Jingyang Yuan, Huazuo Gao, Damai Dai, et al. *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*. 2025. arXiv: [2502.11089](#).

References VIII

- [39] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, et al. *Big Bird: Transformers for Longer Sequences*. 2020. arXiv: [2007.14062](#).